

Generics

Por: Ricardo Cuellar



¿Qué son Generics en Go?

Generics son funciones y tipos que reciben parámetros de tipo. Reusar código sin perder seguridad de tipos.



methods.go

```
func Foo[T any](x T) T  
type Stack[T any] struct{...}
```



¿Qué significa T, K, V y R?

Son convenciones comunes:

- T = Type (El tipo principal)
- K = Key (Llave de algún map)
- V = Value (Valor de algún map)
- R = Result (Tipo de retorno)



Primera función genérica

Entrada y salida del mismo tipo:



methods.go

```
func Identity[T any](v T) T
```



¿Qué es un constraint?

Es una regla que un tipo debe cumplir para poder usarse como T. Tenemos:

- any → cualquiera
- comparable → Usable con == y !=
- constraints personalizados → Tipos numéricos, tipos con métodos.



Comparable

Es un constraint práctico útil para:

- Contains
- IndexOf
- Set
- Keys de map



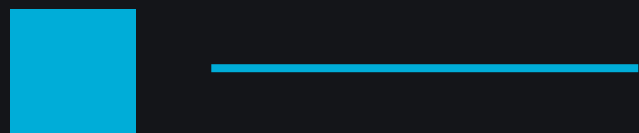
methods.go

```
Contains[T comparable](xs []T, v T) bool
```



Constraints con unión de tipos

Se pueden unir “familias de tipos” como `un: int | int64 | float64`
o más general con `~`



El operador ~ en constraints

Cuando usamos “~” estamos diciendo que vamos a usar ese tipo señalado y cualquier tipo definido por nosotros donde type sea de ese tipo por ejemplo:

~int = Tipos int y todos los que nosotros definimos que sea int.

Ejemplo mental:

- **type UserID int**
- Si tu constraint es int → **UserID no entra**
- Si tu constraint es ~int → **UserID sí entra**



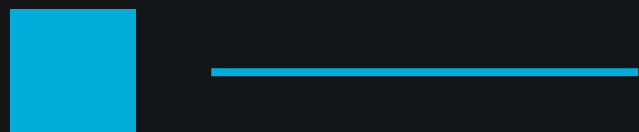
Generics con structs

Cuando usamos **Stack[T]**

La estructura reutilizable quedaría:

- **Stack[int]**
- **Stack[string]**
- **Stack[User]**

Así evitamos tener multiples Stack por tipos



¿Cuándo usar generics?

- La lógica es la misma y solo cambian los tipos.
- Vas a reusar en varios lados.
- Estructuras de datos reutilizables (Stack, Cache, Optional)
- Helpers de slices o maps (Map, Filter, Reduce, GroupBy)

Tip: Si sin generics debemos copiar 3 veces el mismo algoritmo entonces vale la pena usar Generics.



¿Cuándo NO usar generics?

- Solo se usará una vez.
- La abstracción complica mucho la lectura.
- Ya se tienen interfaces claras por comportamiento.
- Estás generalizando por presión social y quieres quedar como el chico o chica PRO.



Ventajas y desventajas de Generics

VENTAJAS:

- Menos duplicación.
- Más seguridad que interface.
- APIs reutilizables y limpias.
- Mejor mantenimiento (en un solo lugar).

DESVENTAJAS:

- Puede bajar la legibilidad del proyecto.
- Firmas más largas y confusas.
- Errores de compilación a veces más “verbosos”.
- Se presta a sobre-abstraer.



Generics vs Interfaces

Generics (tipo): Cuando cambia el tipo de datos, pero la lógica es la misma.

Interfaces (Comportamiento): Cuando quieres depender de métodos y no de tipos.

Nota: Generics no reemplaza interfaces se complementan entre ellas.



Gracias



www.devtalles.com



[/ricardocuellars](https://www.linkedin.com/company/ricardocuellars)



[@ricardocuellars](https://twitter.com/ricardocuellars)

